

Atelier 4 : filtres et rapports à la demande

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Chapitre** : 4, Fonctions avancées · **Version** : 2026-10-23

Avant de commencer

Ce que vous allez construire	`rapports.py`, une petite boîte à outils pour répondre en quelques lignes à n'importe quelle question de la direction : des filtres réglables et combinables, un rapport paramétrable, un regroupement récursif sur plusieurs niveaux
Durée	🟢 Débutant : 4 h · 🔵 Standard : 3 h · 🟣 Expert : 2 h (sans les défis)
Prérequis	Avoir lu le chapitre 4 ; votre dépôt `yonnee-express` (atelier 3)
Fichiers fournis	Aucun nouveau : le registre complet de l'atelier 3
Rendu	Votre dépôt GitHub `yonnee-express`, avec l'étiquette `atelier-4`

A l'atelier 3, chaque statistique avait son propre bout de code. Le problème : à chaque nouvelle question, il faut tout réécrire. Cet atelier change de méthode : on écrit des fonctions **génériques**, qu'on combine. Par exemple :

```
filtrer(livraisons, par_livreur("Awa"), plus_lourd_que(8), entre_dates(date(2026, 1, 1),
        date(2026, 12, 31)))
rapport(livraisons, cle=lambda l: l.livreur, valeur=lambda l: l.duree_min, agregat=moyenne)
grouper(livraisons, lambda l: l.date.year, lambda l: l.quartier)
```

> 📍 **Point de départ** : votre dépôt de l'atelier 3. Créez `rapports.py` ; copiez-y le tuple nommé `Livraison` et la fonction `charger` de `statistiques.py`.

Comment travailler

Mêmes règles que les ateliers précédents. Chaque fonction a une **docstring** (une phrase entre triples guillemets sous le `def`, qui dit ce qu'elle fait).

Étape 1 : les fabriques de conditions (40 min)

🎯 **Objectif** : des fonctions qui **fabriquent** des conditions réglables.

Une **condition** est une fonction qui reçoit une livraison et renvoie `True` ou `False`. Une **fabrique** est une fonction qui reçoit un réglage et **renvoie** une condition (chapitre 4, section 6.2).

🛠️ **A faire** : écrivez ces cinq fabriques, chacune en une ligne avec `return lambda l: ...` :

Fabrique	La condition renvoyée est vraie si…
`dans_quartiers(*quartiers)`	le quartier de la livraison fait partie de ceux donnés
`par_livreur(nom)`	le livreur est `nom`
`plus_lourd_que(kg)`	le poids dépasse `kg`
`entre_dates(debut, fin)`	la date est entre `debut` et `fin`, inclus
`en_retard(seuil=60)`	la durée dépasse `seuil` minutes

✅ **Vérification** : dans le terminal Python, `en_retard()(Livraison(1, date(2026, 1, 5), "Awa", "Yoff", 2.0, 5000, 2000, 75))` renvoie `True`, et `en_retard(90)(...)` sur la même livraison, `False`. `dans_quartiers("Yoff", "Pikine")` accepte une livraison à Yoff.

💡 **A comprendre** : pourquoi `dans_quartiers` a-t-il une étoile, et pas `par_livreur` ? Que contient `quartiers` à l'intérieur de la fonction ?

```
git add .
git commit -m "Fabriques de conditions"
```

Étape 2 : combiner les conditions (25 min)

🛠️ **A faire** : `filtrer(livraisons, *conditions)` renvoie la liste des livraisons qui remplissent **toutes** les conditions. Indice : la fonction `all(...)` renvoie `True` si tous les éléments sont vrais ; une compréhension suffit.

✅ **Vérification**

Appel	Résultat
`filtrer(livraisons, dans_quartiers("Pikine"), en_retard())`	109 livraisons
`filtrer(livraisons, par_livreur("Awa"), plus_lourd_que(8), entre_dates(date(2026, 1, 1), date(2026, 12, 31)))`	54 livraisons
`filtrer(livraisons)` (sans condition)	2 000 livraisons

💡 **A comprendre** : pourquoi `filtrer` sans condition renvoie-t-il **toutes** les livraisons ? Que vaut `all([])` ?

```
git add .
git commit -m "Filtrer avec des conditions combinées"
```

Étape 3 : un rapport paramétrable (45 min)

🎯 **Objectif** : **une seule** fonction pour tous les rapports « par catégorie ».

🛠️ **A faire** : écrivez

```
def rapport(livraisons, cle, valeur=lambda l: l.frais, agregat=sum, titre="Rapport",
            format_valeur="{:,.0f}"):
```

qui :

1. regroupe les **valeurs** (`valeur(l)`) selon la **clé** (`cle(l)`) dans un dictionnaire de listes ;
2. calcule l'**agrégat** de chaque liste (par défaut la somme) avec une compréhension de dictionnaire ;
3. affiche le titre, puis une ligne par groupe, du plus grand au plus petit résultat, le résultat mis en forme avec `format_valeur.format(...)` (et les virgules des milliers remplacées par des espaces) ;
4. renvoie le dictionnaire des résultats.

Écrivez aussi `moyenne(valeurs)`, pour l'utiliser comme agrégat.

✅ **Vérification** : trois rapports, avec la même fonction.

Appel	Premières lignes attendues
`rapport(livraisons, cle=lambda l: l.quartier, titre="Frais par quartier (F)")`	Pikine 1 076 000, Yoff 967 000, Parcelles 894 000
`rapport(livraisons, cle=lambda l: l.livreur, valeur=lambda l: l.duree_min, agregat=moyenne, titre="Durée moyenne par livreur (min)", format_valeur="{:.1f}")`	Ibou 45.6, Awa 45.1, Fatou 45.0, Modou 44.5, Coumba 44.0
`rapport(filtrer(livraisons, entre_dates(date(2026, 1, 1), date(2026, 12, 31))), cle=lambda l: l.date.month, agregat=len, titre="Nombre de livraisons par mois en 2026")`	mois 5 : 100, puis 9 et 4 : 91

💡 **A comprendre**

- Pourquoi `agregat=len` marche-t-il ? Et `agregat=max` ?
- Les durées moyennes des livreurs sont presque identiques (de 44 à 45,6 min). Que cela confirme-t-il sur la question des retards d'Awa (atelier 3) ?

```
git add .
git commit -m "Rapport paramétrable"
```

Étape 4 : regrouper sur plusieurs niveaux, par récursivité (45 min)

🎯 **Objectif** : une fonction qui regroupe par année, puis par quartier, puis par ce qu'on veut, sur **autant de niveaux** qu'on donne de clés.

🛠️ **A faire** : écrivez la fonction récursive `grouper(livraisons, *cles)` :

- **cas de base** : s'il n'y a plus de clé, elle renvoie le **nombre** de livraisons ;
- **cas récursif** : elle regroupe les livraisons selon la **première** clé, puis appelle `grouper` sur chaque groupe avec les clés **suivantes**, et renvoie un dictionnaire (trié par clé).

▼ 💡 Indice : séparer la première clé des suivantes

```
premiere, *suivantes = cles
```

`premiere` reçoit le premier élément du tuple, `suivantes` une liste avec tous les autres (éventuellement vide). Puis `grouper(sous_liste, *suivantes)` .

✅ **Vérification**

```
niveaux = grouper(livraisons, lambda l: l.date.year, lambda l: l.quartier)
print(niveaux[2025])
print(niveaux[2026]["Pikine"])
detail = grouper(livraisons, lambda l: l.livreur, lambda l: l.quartier, lambda l: l.poids > 5)
print(detail["Awa"]["Médina"])
```

```
{'Médina': 217, 'Parcelles': 225, 'Pikine': 223, 'Plateau': 232, 'Yoff': 223}
162
{False: 30, True: 48}
```

💡 **A comprendre** : déroulez à la main `grouper(trois_livraisons, cle_quartier)` sur trois livraisons : combien de fois `grouper` est-elle appelée ? Que renvoie l'appel le plus profond ?

```
git add .
git commit -m "Regroupement récursif sur plusieurs niveaux"
```

Étape 5 : la démonstration (20 min)

🛠️ **A faire** : une fonction `main()` qui charge le registre et affiche les réponses des étapes 2 à 4, puis `if __name__ == "__main__": main()`. Ajoutez **une question à vous**, répondue en une ou deux lignes avec vos outils (par exemple : « le total des frais des colis de plus de 10 kg livrés à Yoff, par livreur »).

✅ **Vérification** : `python rapports.py` affiche tout sans erreur ; votre question tient en deux lignes.

Puis étiquetez :

```
git add .
git commit -m "Démonstration des filtres et rapports"
git tag atelier-4
git push
git push origin atelier-4
```

🏆 Défis (facultatifs)

1. **Le « ou ».** Écrivez `au_moins_une(*conditions)`, une fabrique qui renvoie une condition vraie si **au moins une** des conditions l'est (indice : `any`). Combien de livraisons sont lourdes **ou** en retard ?
2. **La négation.** Écrivez `non(condition)`, qui inverse une condition. Combien de livraisons ne sont **pas** à Pikine ?
3. **Le rapport à deux clés.** Utilisez `rapport` avec `cle=lambda l: (l.livreur, l.quartier)` et `agregat=len`. Que remarquez-vous à l'affichage ? Comment l'améliorer ?
4. 🟣 **Le total récursif.** Écrivez `total(arbre)`, une fonction récursive qui additionne tous les nombres d'un résultat de `grouper`, quel que soit le nombre de niveaux (chapitre 4, section 7). `total(grouper(livraisons, cle1, cle2, cle3))` doit toujours valoir 2 000.

Grille d'autoévaluation

Je sais…	Oui	Pas encore
écrire une fonction qui fabrique et renvoie une autre fonction	☐	☐
accepter un nombre variable de conditions avec `*`	☐	☐
passer des fonctions en paramètre (clé, valeur, agrégat)	☐	☐
donner des valeurs par défaut utiles, sans piège	☐	☐
écrire une fonction récursive avec un cas de base clair	☐	☐
répondre à une nouvelle question en combinant des outils existants	☐	☐